

Giảng viên ra đề: (Chữ ký và Họ tên) Nguyễn Vĩnh Hào	Ngày ra đề: 6/4	Người phê duyệt: (Chữ ký, Chức vụ và Họ tên)	Ngày duyệt đề: .../.../...
---	------------------------	--	-----------------------------------

 TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM KHOA ĐIỆN ĐIỆN TỬ	THI CUỐI KỲ		Học kỳ/năm học 2	2024-2025
			Ngày thi	25/05/2025
	Môn học	Hệ thống điều khiển nhúng		
	Mã môn học	EE3067		
	Thời lượng	90 phút	Mã đề	1
Ghi chú: - Được sử dụng tài liệu				

Câu 1: (L.O.1) (2,5đ)

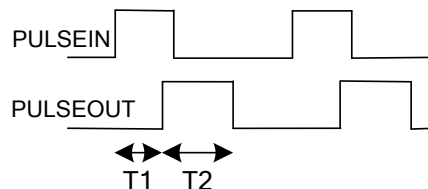
Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (**8-bit địa chỉ A7-A0; 8-bit dữ liệu D7-D0; RD, WR trên 2 chân riêng biệt**) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, kết thúc của module (không giải mã địa chỉ kênh):

- | | |
|---------------------------------|--|
| 1/ Module 1: 16 kênh ADC 12-bit | 4/ Module 4: 16 kênh Encoder 32-bit |
| 2/ Module 2: 10 kênh DAC 12-bit | 5/ Module 5: 8 kênh Digital Input 8-bit |
| 3/ Module 3: 16 kênh PWM 10-bit | 6/ Module 6: 4 kênh Digital Output 8-bit |

Câu 2: (L.O.2) (2,5đ)

Viết chương trình Verilog tên **xuatxung(clk, T1, T2, PULSEIN, PULSEOUT)** thực hiện việc xuất xung ra như sau:

- Đầu vào tín hiệu PULSEIN, đầu ra tín hiệu PULSEOUT
- Đầu vào dữ liệu T1, T2 8 bit đơn vị us
- Đầu vào tín hiệu xung clock clk có tần số 1 MHz.
- Khi có cạnh lên của xung PULSEIN, sau thời gian T1 PULSEOUT = 1, sau thời gian T2 PULSEOUT = 0 như minh họa ở Hình 1.



Hình 1. Giải đồ xung

Câu 3: (L.O.5) (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PA0, PA1, PA2} cho INPUT, {PC6, PC7} cho UART, {PD12, PD13} cho ENCODER, và {PE9, PE11} cho PWM. Thạch anh sử dụng cho STM32F4 là 25MHz.

- Cấu hình các thanh ghi RCC để cho phép CPU chạy ở tần số 168MHz.
- Cấu hình các thanh ghi hệ thống để sử dụng 4 chức năng INPUT, UART, ENCODER, PWM; và UART có khả năng hoạt động DMA.

Câu 4: (L.O.6) (2,5đ)

Trong cấu hình ở Câu 3, các hàm xuất giá trị PWM và nhận dữ liệu từ UART được cho trước:

- Hàm xuất các kênh PWM: void **write_pwm** (uint16_t channel, uint16_t value). Trong đó channel có giá trị từ 1 đến 2 tương ứng với 2 kênh.
- Hàm nhận N byte từ bộ đệm rxbuff qua UART: void **receive_data** (char *rxbuff, int N). Biết dữ liệu rxbuff dạng binary và format chuỗi dữ liệu nhận vào như sau:

0xFF	0xAA	0x04	PWM1	PWM1	PWM2	PWM2
(1byte)	(1byte)	(1byte)	(byteH)	(byteL)	(byteH)	(byteL)

Viết chương trình đọc dữ liệu từ UART, kiểm tra header chuỗi dữ liệu nếu bằng {0xFF, 0xFA} và số byte dữ liệu bằng {0x04} thì chuyển đổi giá trị và xuất ra 2 kênh PWM.

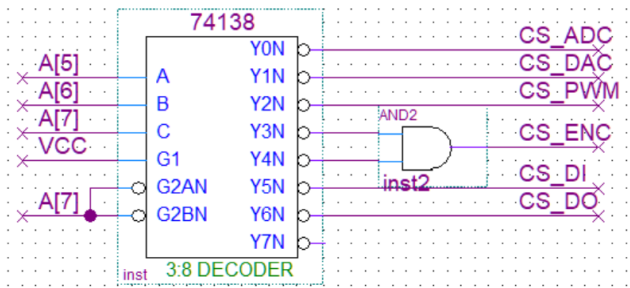
--- HẾT---

ĐÁP ÁN

Câu 1: (2,5đ)

- Vẽ đúng sơ đồ giải mã 74138

(1đ)



- Ghi đúng tâm địa chỉ cho 6 module
- Vẽ đúng sơ đồ kết nối 6 module

(0.5đ)

(1đ)

Câu 2: (2,5đ)

- Khai báo đúng module, đầu vào, đầu ra
- Tính đúng thời gian T1, T2 (đơn vị us)
- Xuất đúng xung PULSEOUT

(0.5đ)

(1đ)

(1đ)

Câu 3: (2,5đ)

a. (0,5đ)

- Cho phép PLL 168MHz:

`RCC_CR |= (1<<16) | (1<<24);`

`RCC_PLLCFGR |= (25<<0) | (336<<6) | (2<<16)`

(0.5đ)

//cho phép HSE, PLL

//M=25, N = 336, P = 2

b. (1,5đ)

- Cấu hình thành ghi xung clock

`RCC_AHB1ENR |= (1<<0) | (1<<2) | (1<<3) | (1<<4);` //clock cho PA, PC, PD, PE

`RCC_AHB1ENR |= (1<<22);`

//clock cho UART6 DMA2

`RCC_APB2ENR |= (1<<5) | (1<<0);`

//clock cho UART6, TIM1

`RCC_APB1ENR |= (1<<2);`

//clock cho TIM4

(0.5đ)

- Cấu hình mode chức năng

`GPIOA_MODER |= (0<<0) | (0<<2) | (0<<4);`

`GPIOC_MODER |= (2<<12) | (2<<14);`

`GPIOB_MODER |= (2<<24) | (2<<26);`

`GPIOE_MODER |= (2<<18) | (2<<22);`

(1.5đ)

`GPIOC_AFR1 |= (8<<24) | (8<<28);`

//UART6 - AF8

`GPIOB_AFR1 |= (2<<16) | (2<<20);`

//TIM4 – ENCODER - AF2

`GPIOE_AFR1 |= (1<<4) | (1<<12);`

//TIM1 – PWM – AF1

`DMA_S1CR |= (5<<25);`

//DMA2, Stream1, Channel5, UART6_RX

`DMA_S2CR |= (5<<25);`

//DMA2, Stream2, Channel5, UART6_TX

Câu 4: (2,5đ)

```
char rxbuff[7];
uint16_t pwmval1, pwmval2;

main(){
    while(1){
        receive_data(rxbuff,7);
        if ((rxbuff[0]==0xFF) && (rxbuff[1]==0xAA) && (rxbuff[2]==0x04))
        {
            pwmval1 = (uint16_t)rxbuff[3]*256 + (uint16_t)rxbuff[4];
            pwmval2 = (uint16_t)rxbuff[5]*256 + (uint16_t)rxbuff[6];
        }
        write_pwm(1,pwmval1);
        write_pwm(2,pwmval2);
    }
}
```